

Noisy Activation Functions

Caglar Gulcehre^{†‡}
 Marcin Moczulski^{†◊}
 Misha Denil^{†◊}
 Yoshua Bengio^{†‡}
^{†‡}University of Montreal

GULCEHRC@IRO.UMONTREAL.CA
 MARCIN.MOCZULSKI@STCATZ.OX.AC.UK
 MISHA.DENIL@GMAIL.COM
 BENGIOY@IRO.UMONTREAL.CA

^{†◊}University of Oxford

Abstract

Common nonlinear activation functions used in neural networks can cause training difficulties due to the saturation behavior of the activation function, which may hide dependencies that are not visible to vanilla-SGD (using first order gradients only). Gating mechanisms that use softly saturating activation functions to emulate the discrete switching of digital logic circuits are good examples of this. We propose to exploit the injection of appropriate noise so that the gradients may flow easily, even if the noiseless application of the activation function would yield zero gradient. Large noise will dominate the noise-free gradient and allow stochastic gradient descent to explore more. By adding noise only to the problematic parts of the activation function, we allow the optimization procedure to explore the boundary between the degenerate (saturating) and the well-behaved parts of the activation function. We also establish connections to simulated annealing, when the amount of noise is annealed down, making it easier to optimize hard objective functions. We find experimentally that replacing such saturating activation functions by noisy variants helps training in many contexts, yielding state-of-the-art or competitive results on different datasets and task, especially when training seems to be the most difficult, e.g., when curriculum learning is necessary to obtain good results.

1. Introduction

The introduction of the piecewise-linear activation functions such as ReLU and Maxout (Goodfellow et al., 2013) units had a profound effect on deep learning, and was a major catalyst in allowing the training of much deeper networks. It is thanks to ReLU that for the first time it was shown (Glorot et al., 2011) that deep purely supervised networks can be trained, whereas using tanh nonlinearity only allowed to train shallow networks. A plausible hypothesis about the recent surge of interest on these piecewise-linear activation

functions (Glorot et al., 2011), is due to the fact that they are easier to optimize with SGD and backpropagation than smooth activation functions, such as sigmoid and tanh. The recent successes of piecewise linear functions is particularly evident in computer vision, where the ReLU has become the default choice in convolutional networks.

We propose a new technique to train neural networks with activation functions which strongly saturate when their input is large. This is mainly achieved by injecting noise to the activation function in its saturated regime and learning the level of noise. Using this approach, we have found that it was possible to train neural networks with much wider family of activation functions than previously. Adding noise to the activation function has been considered for ReLU units and was explored in (Bengio et al., 2013; Nair & Hinton, 2010) for feed-forward networks and Boltzmann machines to encourage units to explore more and make the optimization easier.

More recently there has been a resurgence of interest in more elaborated “gated” architectures such as LSTMs (Hochreiter & Schmidhuber, 1997) and GRUs (Cho et al., 2014), but also encompassing neural attention mechanisms that have been used in the NTM (Graves et al., 2014), Memory Networks (Weston et al., 2014), automatic image captioning (Xu et al., 2015), video caption generation (Yao et al., 2015) and wide areas of applications (LeCun et al., 2015). A common thread running through these works is the use of soft-saturating non-linearities, such as the sigmoid or softmax, to emulate the hard decisions of digital logic circuits. In spite of its success, there are two key problems with this approach.

1. Since the non-linearities still saturate there are problems with vanishing gradient information flowing through the gates; and
2. since the non-linearities only *softly* saturate they do not allow one to take hard decisions.

Although gates often operate in the soft-saturated regime (Karpathy et al., 2015; Bahdanau et al., 2014; Hermann et al., 2015) the architecture prevents them from being fully open or closed. We follow a novel approach to address both of these problems. Our method addresses the second problem through the use of hard-saturating nonlinearities, which allow gates

to make perfectly on or off decisions when they saturate. Since the gates are able to be completely open or closed, no information is lost through the leakiness of the soft-gating architecture.

By introducing hard-saturating nonlinearities, we have exacerbated the problem of gradient flow, since gradients in the saturated regime are now precisely zero instead of being negligible. However, by introducing noise into the activation function which can grow based on the magnitude of saturation, we encourage random exploration.

At test time the noise in the activation functions can be removed or replaced with the expectation, and as our experiments show, the resulting deterministic networks outperform their soft-saturating counterparts on a wide variety of tasks, and allow to reach state-of-the-art performance by simple drop-in replacement of the nonlinearities in existing training code.

The technique that we propose, addresses the difficulty of optimization and having hard-activations at the test time for gating units and we propose a way of performing simulated annealing for neural networks.

Hannun et al. (2014); Le et al. (2015) used ReLU activation functions with simple RNNs. In this paper, we successfully show that, it is possible to use piecewise-linear activation functions with gated recurrent networks such as LSTM and GRU's.

2. Saturating Activation Functions

Definition 2.1. (Activation Function). An activation function is a function $h : \mathcal{R} \rightarrow \mathcal{R}$ that is differentiable almost everywhere.

Definition 2.2. (Saturation). An activation function $h(x)$ with derivative $h'(x)$ is said to right (resp. left) saturate if its limit as $x \rightarrow \infty$ (resp. $x \rightarrow -\infty$) is zero. An activation function is said to saturate (without qualification) if it both left and right saturates.

Most common activation functions used in recurrent networks (for example, $\tanh$ and sigmoid) are saturating. In particular they are soft saturating, meaning that they achieve saturation only in the limit.

Definition 2.3. (Hard and Soft Saturation). Let c be a constant such that $x > c$ implies $h'(x) = 0$ and left hard saturates when $x < c$ implies $h'(x) = 0, \forall x$. We say that $h(\cdot)$ hard saturates (without qualification) if it both left and right hard saturates. An activation function that saturates but achieves zero gradient only in the limit is said to soft saturate.

We can construct hard saturating versions of soft saturating activation functions by taking a first-order Taylor expansion about zero and clipping the results to an appropriate range.

For example, expanding $\tanh$ and sigmoid around 0, with $x \approx 0$, we obtain linearized functions u^t and u^s of $\tanh$ and

sigmoid respectively:

$$\text{sigmoid}(x) \approx u^s(x) = 0.25x + 0.5 \quad (1)$$

$$\tanh(x) \approx u^t(x) = x. \quad (2)$$

Clipping the linear approximations result to,

$$\text{hard-sigmoid}(x) = \max(\min(u^s(x), 1), 0) \quad (3)$$

$$\text{hard-tanh}(x) = \max(\min(u^t(x), 1), -1). \quad (4)$$

The motivation behind this construction is to introduce linear behavior around zero to allow gradients to flow easily when the unit is not saturated, while providing a crisp decision in the saturated regime.

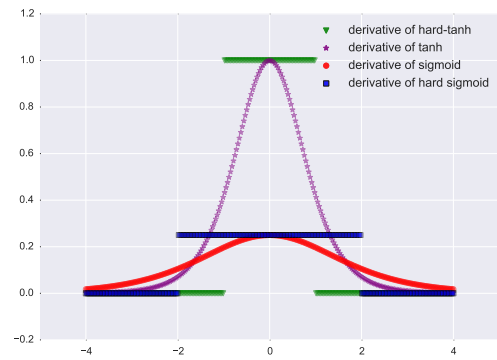


Figure 1. The plot for derivatives of different activation functions.

The ability of the hard-sigmoid and hard-tanh to make crisp decisions comes at the cost of exactly 0 gradients in the saturated regime. This can cause difficulties during training: a small but not infinitesimal change of the pre-activation (before the nonlinearity) may help to reduce the objective function, but this will not be reflected in the gradient.

In the rest of the document we will use $h(x)$ to refer to a generic activation function and use $u(x)$ to denote its linearization based on the first-order Taylor expansion about zero. hard-sigmoid saturates when $x \leq -2$ or $x \geq 2$ and hard-tanh saturates when $x \leq -1$ or $x \geq 1$. We denote the threshold by x_t . Absolute values of the threshold are $x_t = 2$ for hard-sigmoid and $x_t = 1$ for the hard-tanh.

Remark. Let us note that both $\text{hard-sigmoid}(x)$, $\text{sigmoid}(x)$ and $\tanh(x)$ are all contractive mapping. $\text{hard-tanh}(x)$, becomes a contractive mapping only when its input is greater than the threshold. An important difference among these activation functions is their fixed points. $\text{hard-sigmoid}(x)$ has a fixed point at $x = \frac{2}{3}$. However the fixed-point of sigmoid is $x \approx 0.69$. Any $x \in \mathcal{R}$ between -1 and 1 can be the fixed-point of $\text{hard-tanh}(x)$, but the fixed-point of $\tanh(x)$ is 0. $\tanh(x)$ and $\text{sigmoid}(x)$ have point attractors at their fixed-points. Those mathematical differences among the saturating activation functions can make them behave differently with RNNs and deep networks.

The highly non-smooth gradient descent trajectory may bring the parameters into a state that pushes the activations of a unit towards the 0 gradient regime for a particular example, from where it may become difficult to escape and the unit may get stuck in the 0 gradient regime.

When units saturate and gradients vanish, an algorithm may require many training examples and a lot of computation to recover.

3. Annealing with Noisy Activation Functions

Consider a noisy activation function $\phi(x, \xi)$ in which we have injected iid noise ξ , to replace a saturating nonlinearity such as the hard-sigmoid and hard-tanh introduced in the previous section. In the next section we describe the proposed noisy activation function which has been used for our experiments, but here we want to consider a larger family of such noisy activation functions, when we use a variant of stochastic gradient descent (SGD) for training.

Let ξ have variance σ^2 and mean 0. We want to characterize what happens as we gradually anneal the noise, going from large noise levels ($\sigma \rightarrow \infty$) to no noise at all ($\sigma \rightarrow 0$).

Furthermore, we will assume that ϕ is such that when the noise level becomes large, so does its derivative with respect to x :

$$\lim_{|\xi| \rightarrow \infty} \left| \frac{\partial \phi(x, \xi)}{\partial x} \right| \rightarrow \infty. \quad (5)$$

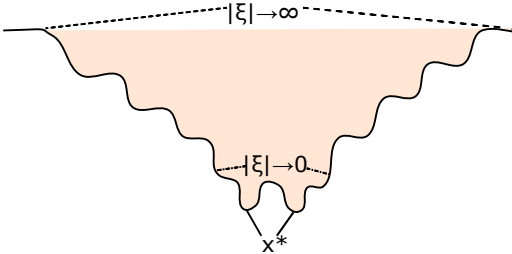


Figure 2. An example of a one-dimensional, non-convex objective function where a simple gradient descent will behave poorly. With large noise $|\xi| \rightarrow \infty$, SGD can escape from saddle points and bad local-minima as a result of exploration. As we anneal the noise level $|\xi| \rightarrow 0$, SGD will eventually converge to one of the local-minima x^* .

In the 0 noise limit, we recover a deterministic nonlinearity, $\phi(x, 0)$, which in our experiments is piecewise linear and allows us to capture the kind of complex function we want to learn. As illustrated in Figure 2, in the large noise limit, large gradients are obtained because backpropagating through ϕ gives rise to large derivatives. Hence, the noise drowns the signal: the example-wise gradient on parameters is much larger than it would have been with $\sigma = 0$. SGD therefore just sees noise and can move around anywhere in parameter space without “seeing” any trend.

Annealing is also related to the signal to noise ratio where SNR can be defined as the ratio of the variance of noise σ_{signal} and σ_{noise} , $SNR = \frac{\sigma_{\text{signal}}}{\sigma_{\text{noise}}}$. If $SNR \rightarrow 0$, the model will do pure random exploration. As we anneal SNR will increase, and when σ_{noise} converges to 0, the only source of exploration during the training will come from the noise of Monte Carlo estimates of stochastic gradients.

This is precisely what we need for methods such as simulated annealing (Kirkpatrick et al., 1983) and continuation methods (Allgower & Georg, 1980) to be helpful, in the context of the optimization of difficult non-convex objectives. With high noise, SGD is free to explore all parts of space. As the noise level is decreased, it will prefer some regions where the signal is strong enough to be “visible” by SGD: given a finite number of SGD steps, the noise is not averaged out, and the variance continues to dominate. Then as the noise level is reduced SGD spends more time in “globally better” regions of parameter space. As it approaches to zero we are fine-tuning the solution and converging near a minimum of the noise-free objective function. A related approach of adding noise to gradients and annealing the noise was investigated in (Neelakantan et al., 2015) as well. Ge et al. (2015) showed that SGD with annealed noise will globally converge to a local-minima for non-convex objective functions in polynomial number of iterations. Recently, Mobahi (2016) propose an optimization method that applies Gaussian smoothing on the loss function such that annealing weight noise is a Monte Carlo estimator of that.

4. Adding Noise when the Unit Saturate

A novel idea behind the proposed noisy activation is that **the amount of noise added to the nonlinearity is proportional to the magnitude of saturation of the nonlinearity**. For $\text{hard-sigmoid}(x)$ and $\text{hard-tanh}(x)$, due to our parametrization of the noise, that translates into the fact that the noise is only added when the hard-nonlinearity saturates. This is different from previous proposals such as the noisy rectifier from Bengio (2013) where noise is added just before a rectifier (ReLU) unit, independently of whether the input is in the linear regime or in the saturating regime of the nonlinearity.

The motivation is to keep the training signal clean when the unit is in the non-saturating (typically linear) regime and provide some noisy signal when the unit is in the saturating regime.

$h(x)$ refer to hard saturation activation function such as the hard-sigmoid and hard-tanh introduced in Sec. 2, we consider noisy activation functions of the following form:

$$\phi(x, \xi) = h(x) + s \quad (6)$$

and $s = \mu + \sigma\xi$. Here ξ is an iid random variable drawn from some generating distribution, and the parameters μ and σ (discussed below) are used to generate a location scale family from ξ .

Intuitively when the unit saturates we pin its output to the